

`Server/Server_handlers.py`

```
# server_handlers.py
# ! 0† 0" 0 „ !† 0f< í ,° /° <

import os
import sys
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from env_utils import RECEIVE_DIR
from ai_services import OPENAI_AVAILABLE, openai_client, analyze_text, generate_quiz
```

```

class ServerCommandHandlers:
    """
    """

    def __init__(self, data_manager, log_callback=None):
        self.data_manager = data_manager
        self._log = log_callback or (lambda msg: print(msg))

    def handle_login(self, data):
        """
        """
        username = data.get("username", "")
        email = data.get("email")
        password = data.get("password", "")

        if not email:
            return {"status": "error", "message": "Email is required"}

        # Check if email is valid
        if not username:
            username = email.split('@')[0]

        # Check if password is valid
        if not password:
            password = ""

        # Folder ID is required
        user = self.data_manager.get_user_by_email_password(email, password)

        if user:
            # User found
            return {
                "status": "success",
                "message": "User logged in successfully",
                "user": {
                    "folder_id": user.get("folder_id"),
                    "user_name": user.get("user_name"),
                    "email": user.get("email")
                }
            }
        else:
            # User not found
            new_folder = self.data_manager.create_user_with_default_folder(
                username=username,
                email=email,
                password=password,
            )

```

Server/server_handlers.py

```

def handle_new_folder(self, data):
    title = "ο'ι ,ι ι ..."
    )
    if new_folder:
        return {
            "status": "success",
            "message": "ι ι <ι 'ι °ι ° -°ι ι ι μ° ° /. Ε,°ο†,ι .° Ε° ι ι -° ι",
            "user": {
                "folder_id": new_folder.get("folder_id"),
                "user_name": new_folder.get("user_name"),
                "email": new_folder.get("email")
            }
        }
    else:
        return {"status": "error", "message": "ι ι <ι 'ι ° -°ι ι ι /ι ι ι ι μ° ° /."}

def handle_save_text(self, data):
    """ι .° ...ι .ι ,ι ,ι ι °†ι ° ι ι /ι ,°¥...ι °†ι ι ι ¥ (File ι ι .°, °§ ι ι <ι ')"""
    filename = data.get("filename")
    text = data.get("text", "")
    keywords = data.get("keywords", "")
    summary = data.get("summary", "")

    if not filename:
        return {"status": "error", "message": "ι ι ...°"ι .ι Ε†μ° ι § ι ι ι ι μ° ° /."}

    if not text:
        return {"status": "error", "message": "ι ι /ι ,Ε° °,,ι .ι ι μ° ° /."}

    try:
        base_name, _ = os.path.splitext(filename)

        # 1)ι °†,ι ι /ι ,ι ι ...ι ι ¥
        text_file = os.path.join(RECEIVE_DIR, f"{base_name}.txt")
        with open(text_file, 'w', encoding='utf-8') as f:
            f.write(text)

        self._log(f"ι ι /ι ,ι ι ¥ι °£ : {text_file}")

        # 2)ι ι %ι ι ... Ε†%°ι ι ι - (ι .° ...ι .ι ,ι ,ι ι ι ι ι ι ι /ι ° /ι ι %ι ι <ι
        summary_file_path = None
        if keywords or summary:
            summary_filename = f"{base_name}_sub.txt"
            summary_file_path = os.path.join(RECEIVE_DIR, summary_filename)
            with open(summary_file_path, 'w', encoding='utf-8') as f:
                if keywords:
                    f.write("[ι /ι ° ]\n")
                    f.write(keywords + "\n\n")
                if summary:
                    f.write("[ι ι %]\n")
                    f.write(summary)

        # File ι ι .°,ι ι ι ι ...° -°ι (folder_id° NULL°ι ι /ι - ° ι ι ...ι ι <ι 'ι ι °ι
        # keywordsι summary° summary_file_path ι ι ...ι °§ ι ι ¥ (ι ι .°, ι ι ><° ...ι ι <ι '
        file_data = self.data_manager.create_file(
            folder_id=None, # ° ι ι ...ι ι <ι 'ι ι ι °ι ι ι ι ,ι .°fl °ι folder_id °¶ ι ι

```

Server/server_handlers.py

```
        file_path=text_file,
        convert_file_path=text_file,
        summary_file_path=summary_file_path
    )

    if not file_data:
        return {"status": "error", "message": "í ì ... ì ì ¥ì /í ¢"}

    file_id = file_data.get('file_id')

    return {
        "status": "success",
        "message": "í ì /í , °° ì °, í ì ... ì ì ¥ì °£ ",
        "file_id": file_id,
        "text_file": text_file
    }
except Exception as e:
    import traceback
    traceback.print_exc()
    return {"status": "error", "message": f"í ì /í , ì ì ¥ì /°¥ : {e}"}

def handle_manage_folder(self, data):
    """í .° ¤. °f< (ERD ¤µ<ì ì: Folder í ì .°, )"""
    action = data.get("action")
    folder_name = data.get("folder_name")
    user_email = data.get("user_email")
    new_folder_name = data.get("new_folder_name")

    # ì <ì 'ì ì °‡. ¤° ì ,ì /¤,°
    user_folders = self.data_manager.get_user_folders(user_email)
    if not user_folders:
        return {"status": "error", "message": "ì <ì 'ì °¥... ì°¾ ì ì ì µ° ° /."}

    first_folder = user_folders[0]
    user_name = first_folder.get('user_name', '')
    password = first_folder.get('password', '')

    if action == "create":
        # ì í .° ì ì - (°¶ °"¤ í .° ° NULL ° ° ¤,°‡, í .° )
        new_folder = self.data_manager.create_folder(
            p_folder_id=None, # ìµ ì ì í .°
            title=folder_name,
            user_name=user_name,
            email=user_email,
            password=password
        )
        if new_folder:
            return {"status": "success", "message": f"í .° '{folder_name}' ¤° ì ì -° ì ì µ°"}
        else:
            return {"status": "error", "message": "í .° ì ì -ì ì /í ¤ í ì µ° ° /."}

    elif action == "update":
        # í .° ì .°f °‡ ¤‡%
        folders = self.data_manager.get_user_folders(user_email)
```

Server/server_handlers.py

```
target_folder = None
for folder in folders:
    if folder.get('title') == folder_name:
        target_folder = folder
        break

if not target_folder:
    return {"status": "error", "message": "í .° º¥... ì°¾ ì ì ì μ° ° /."}

updated = self.data_manager.update_folder(
    target_folder.get('folder_id'),
    title=new_folder_name
)
if updated:
    return {"status": "success", "message": f"í .° º° '{new_folder_name}'°; º‡ º†%°"}
else:
    return {"status": "error", "message": "í .° ì ì ì ì /í ºí ì μ° ° /."}

elif action == "delete":
    # í .° ì >ì
    folders = self.data_manager.get_user_folders(user_email)
    target_folder = None
    for folder in folders:
        if folder.get('title') == folder_name:
            target_folder = folder
            break

    if not target_folder:
        return {"status": "error", "message": "í .° º¥... ì°¾ ì ì ì μ° ° /."}

    # º,ºº‡, í .° ° ì >ì º¶ º°
    if target_folder.get('title') == '' ì ,í ì ...' and len(folders) == 1:
        return {"status": "error", "message": "º,ºº‡, í .° ° ì >ì í ì ì ì μ° ° /."}

    success = self.data_manager.delete_folder(target_folder.get('folder_id'))
    if success:
        return {"status": "success", "message": f"í .° '{folder_name}'º° ì >ì ° ì ì μ°"}
    else:
        return {"status": "error", "message": "í .° ì >ì ì ì /í ºí ì μ° ° /."}

else:
    return {"status": "error", "message": "ì ì ì ° ì ì ì ° ° /."}

def handle_load_file(self, data):
    """í ì ... ° .ì ' °ì ° (ERD ºμ<ì ì°: File í ì .°, )"""
    file_id = data.get("file_id")

    file_data = self.data_manager.get_file(file_id)
    if file_data:
        # í ì ... º†%°ì ì ì í ì /í , ì %º,°
        file_path = file_data.get('file_path') or file_data.get('convert_file_path')
        text = ""
        if file_path and os.path.exists(file_path):
            try:
                with open(file_path, 'r', encoding='utf-8') as f:
```

Server/server_handlers.py

```
def handle_read(self, request):
    except Exception as e:
        self._log(f"í ì ... ì %E,° ì /°¥ : {e}")

    # ì ì % í ì ... E†%° ì ì í /ì ° /ì ì % ì %E,° (summary_file_path í ì ... ì ì %
    keyword_text = ""
    summary_text = ""
    summary_file_path = file_data.get('summary_file_path')
    if summary_file_path and os.path.exists(summary_file_path):
        try:
            with open(summary_file_path, 'r', encoding='utf-8') as f:
                summary_content = f.read()
                # í /ì ° ì ì ì % ì ¶ ì ¶
                if "[í /ì ° ]" in summary_content:
                    parts = summary_content.split("[ì ì %]")
                    keyword_text = parts[0].replace("[í /ì ° ]", "").strip()
                    if len(parts) > 1:
                        summary_text = parts[1].strip()
                elif "[ì ì %]" in summary_content:
                    summary_text = summary_content.replace("[ì ì %]", "").strip()
        except Exception as e:
            self._log(f"ì ì % í ì ... ì %E,° ì /°¥ : {e}")

    # í .ì f ° ° ì .í ° E° ì ,ì /E,° (ì .ì ° μ ì í <í □)
    quizzes = self.data_manager.get_quizzes_by_file(file_id)
    quiz_data = None
    if quizzes:
        quiz_data = {
            "questions": []
        }
        for quiz in quizzes:
            quiz_data["questions"].append({
                "question": quiz.get('question', ''),
                "answer": quiz.get('correct_answer', ''),
                "explanation": quiz.get('explanation', ''),
                "quiz_id": quiz.get('quiz_id'),
                "user_answer": quiz.get('user_answer', '') # ì .ì ° μ ì í <í □
            })

    return {
        "status": "success",
        "file_data": {
            "file_id": file_data.get('file_id'),
            "title": file_data.get('title'),
            "text": text,
            "keywords": keyword_text,
            "summary": summary_text,
            "quiz": quiz_data
        }
    }
else:
    return {"status": "error", "message": "í ì ... ì ì % ì ì ì μ ° ° /."}

def handle_save_score(self, data):
    """ì - ì E†°E†... ì ì ¥ (ERD Eμ<ì ì : Quiz í ì .°, ì correct_is ì ° ° ì .í ,)"""
```

Server/server_handlers.py

```
def handle_quiz_result(self, data):
    quiz_results = data.get("quiz_results", []) # [{"quiz_id": 1, "user_answer": "...", "correct_is": True}]

    if not file_id:
        # 文件ID不能为空
        correct = data.get("correct", 0)
        total = data.get("total", 0)
        self._log(f"文件ID不能为空: {correct}/{total}")
        return {"status": "success", "message": "文件ID不能为空"}

    try:
        updated_count = 0
        for result in quiz_results:
            quiz_id = result.get("quiz_id")
            user_answer = result.get("user_answer", "")
            correct_is = result.get("correct_is", False)

            if quiz_id:
                updated = self.data_manager.update_quiz(
                    quiz_id=quiz_id,
                    user_answer=user_answer,
                    correct_is=correct_is
                )
                if updated:
                    updated_count += 1

        return {
            "status": "success",
            "message": f"成功更新了 {updated_count} 条数据"
        }
    except Exception as e:
        return {"status": "error", "message": f"更新失败: {e}"}

def handle_generate_quiz(self, data):
    """生成新的测验"""
    file_id = data.get("file_id")
    text = data.get("text")
    count = data.get("count", 5)

    if not file_id:
        return {"status": "error", "message": "文件ID不能为空"}

    if not text:
        return {"status": "error", "message": "问题文本不能为空"}

    try:
        # 使用 OpenAI 生成问题
        quiz_data = generate_quiz(text, count)

        # 将生成的测验保存到数据库
        saved_quizzes = []
        for question_data in quiz_data.get("questions", []):
            quiz = self.data_manager.create_quiz(
                file_id=file_id,
                question=question_data.get("question", "")
            )
            saved_quizzes.append(quiz)
```

Server/server_handles.py

```
def handle_quiz(self, data): # OpenAI example
    correct_answer=question_data.get("answer", ""),
    explanation=question_data.get("explanation", ""),
    user_answer=None,
    correct_is=False
)
if quiz:
    saved_quizzes.append(quiz)

    return {
        "status": "success",
        "quiz": quiz_data,
        "message": f"{len(saved_quizzes)} quizzes saved."
    }
except Exception as e:
    import traceback
    traceback.print_exc()
    return {"status": "error", "message": f"Error: {e}"}

def handle_generate_single_quiz(self, data):
    """If file_id is not provided, DB is empty"""
    file_id = data.get("file_id")
    question = data.get("question")
    answer = data.get("answer")
    explanation = data.get("explanation")

    if not file_id:
        return {"status": "error", "message": "file_id is required."}

    if not question or not answer:
        return {"status": "error", "message": "question and answer are required."}

    try:
        # Quiz is created
        quiz = self.data_manager.create_quiz(
            file_id=file_id,
            question=question,
            example=None,
            correct_answer=answer,
            explanation=explanation,
            user_answer=None,
            correct_is=False
        )

        if quiz:
            return {
                "status": "success",
                "quiz_id": quiz.get('quiz_id'),
                "message": f"Quiz {quiz_id} created successfully."
            }
        else:
            return {"status": "error", "message": "Quiz creation failed."}
    except Exception as e:
        import traceback
        traceback.print_exc()
```

```
Server/server_handlers.py: error", "message": f"í ·í í ì ¥ í /°¥ : {e}"}
```

```

def handle_load_quizzes(self, data):
    """í ì ... ID° ì Æ±μ° ì§ ì ì ì μ° ° /."}

    file_id = data.get("file_id")

    if not file_id:
        return {"status": "error", "message": "í ì ... ID° ì Æ±μ° ì§ ì ì ì μ° ° /."}

    try:
        quizzes = self.data_manager.get_quizzes_by_file(file_id)
        return {
            "status": "success",
            "quizzes": quizzes
        }
    except Exception as e:
        return {"status": "error", "message": f"í ì ... ì /°¥ : {e}"}

def handle_list_files(self, data):
    """°"α° í ì ... °"'°ì ìì°í ""

    try:
        files = self.data_manager.get_all_files()
        return {
            "status": "success",
            "files": files
        }
    except Exception as e:
        return {"status": "error", "message": f"í ì ... °"'°ì ìì°í ì /°¥ : {e}"}

def handle_analyze_text(self, data):
    """í ì /í , °¶ ì (í /ì ° /ì ì %)"""
    analysis_type = data.get("type")
    text = data.get("text")

    try:
        result = analyze_text(text, analysis_type)
        return {
            "status": "success",
            "type": analysis_type,
            "result": result
        }
    except Exception as e:
        return {"status": "error", "message": f"°¶ ì ì /°¥ : {e}"}

def handle_convert_audio(self, data):
    """ì /° ì / í ì ... °± í (STT) - ì °† ì ì ì† °f< (OpenAI Whisper API ì <ì ')"

    from ai_services import STT_AVAILABLE, STT_ERROR, transcribe_audio

    filename = data.get("filename")
    file_path = os.path.join(RECEIVE_DIR, filename)

    if not os.path.exists(file_path):
        return {"status": "error", "message": f"í ì ... ì°¼ ì ì ì μ° ° /: {filename}"}

    if not STT_AVAILABLE:

```


Server/server_handlers.py

```
Server/server_handlers.py
if STT_ERROR:
    error_msg += f"\n{STT_ERROR}"
error_msg += "\n\n"
error_msg += "\n1. OpenAI API key not found in .env file, please set it up."
error_msg += "\n2. openai package not installed, please run: pip install openai"
return {"status": "error", "message": error_msg}

try:
    # OpenAI Whisper API
    self._log(f"Processing audio file: {filename} (OpenAI Whisper API)")
    text = transcribe_audio(file_path)

    # Save transcription to file
    text_file = os.path.join(RECEIVE_DIR, f"{os.path.splitext(filename)[0]}.txt")
    with open(text_file, 'w', encoding='utf-8') as f:
        f.write(text)

    self._log(f"Transcription completed for {filename} -> {len(text)} characters")

    return {
        "status": "success",
        "message": "Transcription completed",
        "text": text,
        "text_file": text_file
    }
except Exception as e:
    error_msg = f"Exception: {e}"
    self._log(f"Exception occurred while processing {filename}: {e}")
    return {"status": "error", "message": error_msg}
```